

Week13_예습과제_권지원

Chapter 08. 텍스트 분석 (8.6~8.9)

8.6 토픽 모델링 (Topic Modeling)

토픽 모델링의 개념 및 수학적 배경

- 방대한 문서 집합(Corpus)에 잠재되어 있는 숨은 핵심 주제(Topic)를 찾아내기 위한 비지도학습 기법
- 문서를 핵심 주제를 요약할 수 있는 중심 단어(Word)들의 확률 분포로 변환함
- 대량의 문서 분류 비용을 절감하며 검색 엔진, 추천 시스템, 트렌드 분석 등에 핵심적으로 활용됨

LDA (Latent Dirichlet Allocation) 알고리즘 심화

- **알고리즘 개요:** 토픽 모델링의 가장 대표적인 알고리즘으로, 디리클레 분포(Dirichlet Distribution)를 활용한 확률적 생성 모델
 - (주의) 차원 축소 기법인 선형 판별 분석(Linear Discriminant Analysis)과 약어만 동일하며 완전히 다른 알고리즘임
- **기본 가정 (Generative Process):** - "하나의 문서는 여러 개의 잠재 토픽(Topic)이 혼합되어 구성됨"
 - "하나의 토픽은 여러 단어(Word)들의 확률 분포로 구성됨"
- 모델은 문서 내에 실제 사용된 단어 배열을 관찰하여, 어떤 잠재 토픽들의 조합에 의해 해당 문서가 생성되었는지를 확률적으로 역추론(Reverse Engineering)함

실습 파이프라인 (20 뉴스그룹 데이터)

- **피처 벡터화의 제약:** - LDA 알고리즘은 단어의 순수 등장 '빈도수(Count)' 자체를 확률 추론의 기본 척도로 삼음
 - 가중치가 인위적으로 조정된 소수점 형태의 TF-IDF 행렬은 사용 불가하며, 반드시 **Count 기반 벡터화**(`CountVectorizer`)만 적용해야 함
- **모델 학습 및 핵심 단어 추출 기법:**
 - `LatentDirichletAllocation(n_components=k)` 모델 생성을 통해 목표 토픽 개수 k 를 사전에 명시하여 학습을 수행함
 - `components_` 속성 구조: 반환 배열의 크기는 (토픽 개수, 전체 단어 피처 개수) 형태를 취함
 - 각 배열 내부의 수치는 특정 토픽 공간 내에서 해당 단어 피처가 가지는 연관도 값(할당 횟수 기반의 확률 지표)을 의미함
 - **핵심 키워드 매핑:** NumPy의 `argsort()[::-1]` 메서드를 통해 연관도 수치가 높은 순서대로 인덱스를 추출한 뒤, `CountVectorizer.get_feature_names()` 와 결합하여 토픽별 중심 키워드를 직관적으로 추출함

8.7 문서 군집화 (Document Clustering)

문서 군집화의 개념과 차별점

- 유사한 시맨틱(의미)과 구조적 텍스트 구성을 가진 문서들을 동일한 카테고리로 묶는 비지도학습 기법
- 텍스트 분류(Classification)와 목적은 유사하나, 사전에 정의된 카테고리나 학습용 정답 레이블 (Target Label) 없이 데이터 자체의 피쳐 벡터 간 유사도 거리에만 의존하여 클러스터링을 자발적으로 수행한다는 차이점이 존재함

Opinion Review 데이터 실습 파이프라인

- **데이터 로드 및 통합:** 51개의 개별 리뷰 텍스트 파일(호텔, 자동차, 전자기기 등)을 파이썬의 `glob` 모듈과 `for` 루프를 순회하며 파일명과 내용을 하나의 DataFrame 형태로 통합함
- **커스텀 어근 추출과 피쳐 벡터화 결합:** - `TfidfVectorizer` 적용 시 내부 `tokenizer` 파라미터에 사용자 정의 어근 변환 함수(Lemmatization을 수행하는 `LemNormalize()`)를 콜백으로 전달함
 - 이를 통해 텍스트 데이터의 정규화 가공과 벡터화 처리를 단일 프로세스로 동시에 처리함
- **K-Means 군집화 적용:** `KMeans(n_clusters=3)` 등을 설정하여 전체 리뷰 문서들을 상호 배타적인 3개의 클러스터(포터블 전자기기, 자동차, 호텔 등)로 자동 분류함

`cluster_centers_`를 활용한 군집별 핵심 단어 (Centroid Analysis)

- K-평균 군집화 완료 후 모델의 내장 속성인 `cluster_centers_`를 분석하여 각 군집을 관통하는 핵심 단어를 추출함
- **해석 방법:**
 - 반환되는 ndarray 배열의 크기는 (`군집 개수`, `전체 피쳐 개수`) 구조를 가짐
 - 배열 내 개별 값은 각 단어 피쳐가 해당 군집의 중심점(Centroid) 공간을 기준으로 얼마나 밀접하게 위치하는지를 나타내는 **0~1 사이의 상대 좌표 값**임
 - 수치가 **1에 가까울수록** 해당 군집의 시맨틱을 결정짓는 핵심 단어(Top Feature)임을 의미함 (예: 호텔 군집 -> room, hotel, service / 전자기기 군집 -> screen, battery, battery life)
 - `argsort()[::-1]`을 통해 각 행(군집)별로 중심점 거리가 가장 가까운 피쳐 인덱스 순서대로 정렬하여 추출함

8.8 문서 유사도 (Document Similarity)

코사인 유사도 (Cosine Similarity)의 원리

- 두 벡터의 절대적인 크기(Magnitude)나 거리가 아닌, 상호 방향성(Direction)이 얼마나 일치하는지에 기반하여 두 벡터 간의 사잇각(Angle) θ 를 측정하는 지표
- 값이 1에 가까울수록 두 문서의 단어 패턴 방향이 완벽히 일치함을 의미하며, 0은 상호 직교(관련성 없음), -1은 완전히 상반된 패턴임을 가리킴

수학적 공식 및 L2 정규화

- 두 문서 벡터 A 와 B 의 내적(Dot product) 연산 결과를 각 벡터 크기의 곱(L2 Norm)으로 나누어 정규화 과정을 수행함

$$\text{Cosine Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

유클리드 거리 대신 코사인 유사도를 절대적으로 선호하는 이유

기존 거리 지표의 문제점	코사인 유사도의 해결 방식
차원의 저주 (Curse of Dimensionality)	텍스트 피처 벡터화 시 단어 사전에 의해 수만 차원의 고차원 희소 행렬(Sparse Matrix)이 생성됨. 이러한 고차원 공간 내에서는 유클리드 거리 기반 지표의 정확도가 무력화되지만, 코사인 유사도는 각도 기반이므로 패턴의 유사성을 정확히 포착함
문서 길이에 따른 빈도 수 왜곡 (Length Bias)	특정 문서의 길이가 비정상적으로 길어 단순 단어 빈도수(Count)가 압도적으로 많은 경우, 공정한 비교가 불가능함. 코사인 유사도는 분모에 각 벡터의 크기(L2 Norm)를 적용함으로써 스케일을 정규화하여 문서 길이 편향에 따른 왜곡을 완전히 상쇄함

사이킷런 코사인 유사도 API (`cosine_similarity`)

- `sklearn.metrics.pairwise.cosine_similarity` 라이브러리를 활용함
- **희소 행렬(Sparse Matrix) 구조를 입력 파라미터로 직접 수용**하므로, 대용량 메모리 오버헤드가 발생하는 밀집 행렬 변환(`.todense()`) 절차 없이 고속으로 대규모 문서 유사도 연산 처리가 가능함
- 반환값은 모든 입력 문서 쌍(Pair) 간의 유사도 수치를 매핑한 2차원 `ndarray` 형태로 출력됨 (예: 3개 문서 비교 시 (3, 3) 행렬 반환)

8.9 한글 텍스트 처리 (KoNLPy) - 네이버 영화 평점 감성 분석

한글 NLP 처리의 특수성과 형태소 분석의 난이도

- 영어를 비롯한 라틴어 계열 언어와 비교 시, 구조적 특성으로 인해 텍스트 정규화 및 분석 알고리즘 적용 난이도가 매우 높음
- **1. 띄어쓰기(Spacing)의 높은 민감도:** 영어는 띄어쓰기 오류 발생 시 단순 오타자(Out-of-Vocabulary)로 처리되나, 한글은 띄어쓰기 위치에 따라 문장 전체의 시맨틱(의미 구조)이 완전히 왜곡되는 현상이 발생함 (예: "아버지가 방에 들어가신다" vs "아버지 가방에 들어가신다")
- **2. 교착어 특성과 조사 분리의 복잡성:** 한글은 명사나 대명사 등의 어근 뒤에 문법적 관계를 나타내는 조사가 결합되는 형태를 취함 (예: 집은, 집이, 집으로, 집에서). 문맥의 의미를 보존하면서 형태소 분석을 통해 조사를 정교하게 탈락시키고 불용어를 제거하는 전처리 연산의 복잡성이 대단히 높음

KoNLPy (코엔엘파이) 개요 및 형태소 엔진 비교

- 파이썬 개발 환경 내에서 한글 형태소 분석(Morphological Analysis)을 수행하기 위한 표준 패키지
- 말뭉치(Corpus) 데이터를 분석하여 의미를 가지는 최소 단위인 '형태소'로 분해하고, 개별 형태소에 품사 태깅(POS Tagging)을 부착하는 역할을 전담함
- 기존 C, C++, Java 기반의 검증된 오픈소스 한글 형태소 엔진들을 파이썬 래퍼(Wrapper) 기반 인터페이스로 통합 제공함

형태소 분석 엔진 (클래스)	특장점 및 한계	실무 활용 포인트
Twitter / Okt	띄어쓰기 규격이 불분명한 구어체, 이모티콘, 오타, 비속어가 혼재된 SNS형 비정형 텍스트 처리에 유연하며 처리 속도가 빠름	네이버 영화 평점, 블로그 및 소셜 미디어 크롤링 데이터 분석
Kkma (꼬꼬마)	형태소 분석 및 품사 분류 체계의 세분화 수준이 매우 깊고 정밀하나, 형태소 분리 속도가 현저히 느려 대용량 실시간 처리에 불리함	뉴스 기사, 공공 문서, 학술 논문 등 정형 텍스트 정밀 마이닝

Mecab (은전한 낱)	C++ 컴파일 기반으로 작동하여 연산 속도가 압도적으로 우수하나, 기본적으로 Windows 환경에서의 기본 설치 및 구동이 미지원됨 (리눅스/Mac 환경 최적화)	대규모 엔터프라이즈 환경 및 실시간 스트리밍 한글 데이터 파이프라인
----------------------	--	---------------------------------------

KoNLPy 설치 및 구동 환경 (Windows 기준 핵심 주의사항)

- KoNLPy 패키지는 내부적으로 자바 기반 형태소 엔진들을 호출하므로 **Java 가상 머신(JVM)** 환경 인터페이스 구축이 강제됨
- **JDK 설치:** 시스템 비트 수(64bit 등)와 완벽히 일치하는 Java JDK (1.7 버전 이상) 설치가 선행되어야 함
- **JPyype1 설치:** 파이썬 메모리 영역에서 자바 런타임 클래스를 상호 호출하기 위한 브리지 모듈인 `JPyype1` 패키지 설치가 필수적임
- **JAVA_HOME 경로 설정 매커니즘:** 일반적인 자바 개발 환경 설정과 다르게, JDK 최상위 Root 디렉터리가 아니라 가상 머신 구동 파일인 `jvm.dll` 파일이 실제 존재하는 하위 경로(예: `.../bin/server` 폴더)를 시스템 환경 변수의 `JAVA_HOME` 으로 매핑해주어야 엔진 로드 에러가 방지됨

실습: 네이버 영화 평점 데이터 지도학습 감성 분석

- **데이터 로딩 및 인코딩:** 한글 데이터의 완성형 글자 깨짐 현상을 원천 방지하기 위해 `pd.read_csv()` 호출 시 `encoding='cp949'` (혹은 데이터 저장 형식에 따라 `utf-8`) 파라미터를 엄격하게 지정하여 데이터 프레임 생성함
- **클렌징 및 정규화:** 문맥 판단에 무의미한 숫자 데이터를 정규 표현식 모듈인 `re.sub(r"\d+", " ", x)` 를 활용해 일괄 공백으로 변환하고, 결측치는 `.fillna('')` 메서드로 전처리함
- **형태소 토크나이저 구축:** `Twitter()` 객체를 인스턴스화한 후 문장 문자열 유입 시 `morphs()` 메서드를 적용하여 개별 형태소 단어들이 담긴 리스트 객체를 반환하는 커스텀 형태소 분석 함수 `tw_tokenizer(text)` 를 작성함
- **TF-IDF 변환 및 분류 파이프라인 적용:**
 - `TfidfVectorizer` 의 내장 속성인 `tokenizer` 인자에 사전에 정의한 `tw_tokenizer` 함수를 대입함
 - 한글 문맥의 의미 보존력을 높이기 위해 파라미터를 `ngram_range=(1, 2)` 로 확장 설정하여 희소 행렬을 생성함
 - 벡터화 처리가 완료된 행렬 데이터를 `LogisticRegression` 분류기에 주입하고 `GridSearchCV` 교차 검증을 통해 규제 파라미터 `C` 의 최적 가중치를 튜닝 및 학습함
- **테스트 데이터 세트 검증 시의 절대 규칙 (매우 중요):**
 - 검증용 테스트 데이터의 피쳐 벡터화 변환 처리 시, 반드시 학습 데이터(Train Set) 기반으로 이미 `fit()` 처리가 완료된 기존 `TfidfVectorizer` 객체를 재사용하여 `transform()` 만을 호출해야 함
 - 테스트 데이터 세트에 대해 `fit_transform()` 을 중복 수행할 경우, 테스트 데이터 내 고유 형태소 사전을 기반으로 칼럼(피쳐 차원 개수)이 재생성되므로 기존 학습 모델과의 차원 불일치로 인해 예측 연산 수행 시 치명적인 행렬 오류가 발생함